

Learning Ultralytics YOLO, step by step

A sequenced path through docs.ultralytics.com: what the library does, where it fits, how to use each part, and which model to pick for which job. Written against the docs as of September 2026 (YOLO26 is the current release; YOLO27 is announced but not out).

00	How to use this guide	09	Training
01	What Ultralytics is	10	Evaluating and improving
02	Prerequisites	11	Export and deploy
03	Install and first run	12	Tracking and Solutions
04	Tasks: what you want	13	Platform and other surfaces
05	Modes: what you do	14	Licensing
06	Reading results	15	Capstone project
07	Choosing a model	16	Roadmap and checklist
08	Preparing a dataset		

00 How to use this guide

Goal: know what "learning it deeply" means here, so you don't try to read the whole docs site.

The Ultralytics docs are large. Rather than reading them front to back, learn the one pattern everything follows, then go deep on the parts your projects need. Every step below has a

goal, the core idea, working code, and a "try it" exercise. Do the exercises; reading alone won't stick.

caution 0.95

Two things the docs mention briefly that matter a lot in practice. First, licensing (step 14): the default license is AGPL-3.0, which has real consequences for commercial software. Second, YOLO models locate and classify things in images. They don't read text, reason about documents, or replace an LLM.

01 What Ultralytics is

Goal: hold the whole system in your head as one picture.

Ultralytics is a Python package (`pip install ultralytics`) and a command-line tool (`yolo`) for real-time computer vision. It wraps a family of models, mainly YOLO ("You Only Look Once"), plus third-party models like SAM and RT-DETR, behind one consistent interface.

The one grammar to memorize

```
yolo [TASK] MODE ARGS

# examples
yolo detect train    data=my.yaml model=yolo26n.pt epochs=100 imgsz=640
yolo segment predict model=yolo26s-seg.pt source=photo.jpg
yolo export model=best.pt format=onnx
```

Copy

The docs are organized around this grammar:

- **Task** is *what* you want from an image: boxes, masks, keypoints, a label, depth.
- **Mode** is *how* you use the model: train, val, predict, export, track, benchmark.
- **Args** configure the run: which model, image size, epochs, device, and so on.
- **Solutions** are ready-made apps (counting, heatmaps, alarms) that skip Task and Mode.

Three surfaces, same models

- **Python package / CLI:** where you'll spend most of your time.
- **Ultralytics Platform:** a web product for annotating data, training in the cloud, and deploying, without writing code.
- **Ultralytics Inference (Rust):** runs exported models without a Python runtime.

Where it's actually used

Manufacturing defect detection, retail shelf and footfall analytics, traffic and parking monitoring, security and intrusion alerts, sports and fitness pose analysis, agriculture (fruit counting, disease spotting), drone and satellite imagery (rotated boxes), medical imaging research, and document layout detection (finding tables, stamps, title blocks, signatures).

02 Prerequisites

Goal: make sure you're not blocked by something unrelated to YOLO.

- **Python** at a working level: virtual environments, pip, lists, dicts, loops, classes.
- **NumPy basics:** arrays, shapes, slicing. Results come back as tensors or arrays.
- **A little PyTorch:** know what a tensor and a device (`cpu` , `cuda:0` , `mps`) are. You don't need to write training loops yourself.
- **Hardware:** CPU is fine for inference and tiny experiments. For real training you want an NVIDIA GPU. Google Colab or Kaggle notebooks are a free way to get one; the docs link a ready-made Colab tutorial.
- **Vocabulary:** bounding box, confidence score, IoU (overlap between two boxes), NMS (removing duplicate boxes), epoch, train/val/test split, overfitting.

03 Install and first run

Goal: get a prediction on screen in under ten minutes.

```
python -m venv .venv
source .venv/bin/activate
```

```
# Windows: .venv\Scripts\activate
```

Copy

```
pip install ultralytics
```

```
# CLI: weights and the sample image download automatically
yolo predict model=yolo26n.pt source='https://ultralytics.com/images/bus.jpg'
```

The annotated image lands in `runs/detect/predict`. The same thing in Python:

```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")          # COCO-pretrained, 80 classes
results = model("https://ultralytics.com/images/bus.jpg")
results[0].show()                   # or results[0].save("out.jpg")
```

Copy

try it 0.99

Run prediction on a photo from your phone, a folder of images

(`source=path/to/folder`), a video file, and your webcam (`source=0`). Then change `conf=0.5` and see how many detections disappear.

04 Tasks: what you want from an image

Goal: given any vision problem, name the right task in one second.

The model filename suffix picks the task: nothing for detection, `-seg`, `-sem`, `-depth`, `-cls`, `-pose`, `-obb`.

Task	Output	Use it when	Example
Detect	Boxes + class + confidence	You need to know what is where, and rough location is enough	Counting cars, finding title blocks on drawings
Instance segmentation	A pixel mask per object	Exact shape matters, or objects overlap	Measuring crack area, cutting products out of photos

Task	Output	Use it when	Example
Semantic segmentation	A class for every pixel, with no separate objects	You care about regions, not individual items	Road vs sidewalk, floor vs wall
Depth estimation	Distance in meters per pixel, from one photo	You need rough 3D without a depth camera	Robot obstacle distance, room layout
Classify	One label for the whole image	The image is the unit; location doesn't matter	Is this page a cover sheet or a plan sheet?
Pose	Keypoints (joints or landmarks)	You need body or object landmarks	Workout rep counting, posture checks
OBB (oriented boxes)	Rotated boxes with an angle	Objects appear at arbitrary rotations	Aerial ships and vehicles, rotated text blocks

Tracking isn't a separate task. It runs on top of detect, segment, pose, or OBB predictions across video frames (step 12).

rule of thumb 0.9

Start with the cheapest task that answers your question. Classification labels are fastest to create, then boxes, then masks. Only move up when the simpler output genuinely can't do the job.

05 Modes: what you do with a model

Goal: know the six verbs and when each comes into a project.

Mode	What it does	Python
Train	Fits the model to your labeled data	<code>model.train(data=..., epochs=...)</code>

Mode	What it does	Python
Val	Measures accuracy on a validation set	<code>model.val()</code>
Predict	Runs the model on new images or video	<code>model.predict(source=...)</code> or <code>model(source)</code>
Export	Converts to ONNX, TensorRT, OpenVINO, CoreML, and others	<code>model.export(format="onnx")</code>
Track	Predict plus persistent IDs across frames	<code>model.track(source="video.mp4")</code>
Benchmark	Exports to many formats and compares speed and accuracy	<code>from ultralytics.utils.benchmarks import benchmark</code>

A typical project runs in this order: predict with a pretrained model (to check feasibility), then train, val, predict on real data, export, and benchmark on the target hardware.

Arguments you'll use constantly

- `imgsz`: input resolution (default 640). Larger helps with small objects but costs speed.
- `conf`: minimum confidence to keep a prediction. `iou`: overlap threshold for NMS.
- `device`: `cpu`, `0`, `0,1` (multi-GPU), or `mps` on Apple silicon.
- `classes=[0,2]`: keep only certain classes. `save=True`, `save_txt=True`, `save_crop=True`: write outputs to disk.
- `stream=True`: yields results one at a time, so long videos don't use up your memory.

The full list is on the Configuration page (docs → Usage → cfg).

06 Reading results in code

Goal: turn predictions into data your program can act on.

Predict returns a list of `Results` objects, one per image or frame.

```

from ultralytics import YOLO
model = YOLO("yolo26n.pt")

for r in model("street.jpg"):
    print(r.names)                                # {0: 'person', 1: 'bicycle', ...}
    for box in r.bboxes:
        x1, y1, x2, y2 = box.xyxy[0].tolist()    # pixel corners
        cls_id = int(box.cls[0])
        score = float(box.conf[0])
        print(r.names[cls_id], round(score, 2), (x1, y1, x2, y2))

# other task outputs, when the model provides them:
# r.masks.xy          -> polygon outlines (segmentation)
# r.keypoints.xy      -> keypoint coordinates (pose)
# r.obb.xywhr         -> center, size, rotation (OBB)
# r.probs.top1        -> best class index (classification)

annotated = r.plot()                               # NumPy image with drawings

```

Boxes are also available normalized (`xyxyn` , `xywhn` , scaled 0 to 1), which is useful when you map coordinates back onto a differently sized original, such as a PDF page.

try it 0.98

Write a script that counts people in every image in a folder and writes `filename,`
`count` to a CSV.

07 Choosing a model

Goal: pick the family first, then the size.

Which family

Your situation	Use	Why
New project, any standard task	YOLO26	Current release; the only family covering all seven tasks; optional NMS-free output

Your situation	Use	Why
You want the most mature, widely deployed option	YOLO11	Stable, with pretrained weights for detect, segment, classify, pose, and OBB
Existing YOLOv8 or YOLOv5 pipeline	Keep it, then trial YOLO26	Same API, so switching is mostly a filename change. Re-validate before you switch.
Classes you can't list ahead of time, or no training data yet	YOLOE (YOLOE-26) or YOLO-World	Open vocabulary: describe objects with text prompts
Precise masks from a click, box, or text prompt	SAM 3 / SAM 2	Promptable segmentation. SAM 2 and SAM 3 also track through video. SAM 3 weights need access approval on Hugging Face.
Promptable masks on weak hardware	MobileSAM or FastSAM	Lighter alternatives to SAM
You want a transformer detector to compare against	RT-DETR	Baidu's real-time DETR
Auto-labeling a new dataset	SAM or YOLOE to draft labels, then correct by hand	Cuts annotation time
Research or reproducing papers	YOLOv10, YOLO12, YOLOv9	Supported, but not the production recommendation

Skip YOLOv4 (documentation reference only) and YOLOv7 (predict-only, from exported files). YOLO-NAS is no longer maintained by its original authors. YOLO27 isn't released yet, so don't plan around it.

Which size: n, s, m, l, x

YOLO26 detection on COCO at 640 px, from the docs (speed with the NMS-free head):

Size	mAP 50-95	CPU ONNX (ms)	T4 TensorRT (ms)	Params (M)	Good for
n (nano)	40.9	38.9	1.7	2.4	Phones, Raspberry Pi, CPU servers, prototyping
s (small)	48.6	87.2	2.5	9.5	Best default for most real projects
m (medium)	53.1	220.0	4.7	20.4	GPU servers where accuracy matters
l (large)	55.0	286.2	6.2	24.8	High accuracy with a GPU
x (extra large)	57.5	525.8	11.8	55.7	Offline or batch jobs where speed matters least

COCO scores are only a guide. On your own data with a few classes, the gap between n and x is often smaller, so train `s` first and only size up if validation says you need to.

YOLO26 detail worth knowing

YOLO26 detection has two heads. The default uses NMS and is slightly more accurate. Passing `nms=False` switches to the end-to-end head, which returns at most 300 detections without NMS. That's simpler to deploy, especially on edge devices. There are also architecture-only `yolo26-p2.yaml` (for small objects) and `yolo26-p6.yaml` (for large inputs) configs that you train yourself; no pretrained weights are published for them.

08 Preparing a dataset

Goal: produce a dataset YOLO can train on, and avoid the mistakes that ruin most first attempts.

YOLO detection format

```
dataset/
  images/train/  images/val/
```

Copy

```
labels/train/ labels/val/      # one .txt per image, same filename

# labels/train/page_001.txt  (one line per object, values normalized 0-1)
# class x_center y_center width height
0 0.842 0.911 0.290 0.150
1 0.905 0.958 0.070 0.030
```

```
# data.yaml
path: /abs/path/dataset
train: images/train
val: images/val
names:
  0: title_block
  1: sheet_number
```

Copy

Segmentation labels use polygon points instead of boxes. OBB labels use four corner points. Classification needs no label files, just one folder per class. Each task page in the docs shows its exact format.

Where labels come from

Ultralytics Platform (built-in annotation), Roboflow, CVAT, and Label Studio can all export in YOLO format. Converters exist for COCO JSON and other formats (see Guides and Datasets).

Mistakes that matter more than model choice

- **Near-duplicates leaking between train and val.** Pages from the same PDF or frames from the same video should all go in the same split. Otherwise validation scores look better than real-world performance.
- **Inconsistent labeling.** If two people draw "title block" differently, the model learns the noise. Write a one-page labeling guide.
- **Not enough variety.** 300 varied images beat 3,000 near-identical ones. Cover different sources, layouts, lighting, and rotations.
- **No background images.** Include some images with nothing to detect (empty label file) to reduce false positives.

try it 0.97

Train on the built-in `coco8.yaml` (8 images, downloads automatically) and open the folder it creates, so you can see exactly how a valid dataset is laid out.

09 Training a custom model

Goal: run a proper fine-tune and understand what's happening while it runs.

```
from ultralytics import YOLO

model = YOLO("yolo26s.pt")  # start from pretrained weights (transfer learn
model.train(
    data="data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,          # or -1 to auto-fit GPU memory
    patience=20,        # stop early if no improvement
    device=0,
    project="runs", name="exp1",
)
```

Copy

Always start from a pretrained `.pt` file unless you have a very large dataset. Starting from a `.yaml` trains from scratch, which needs far more data and time.

What the run folder gives you

- `weights/best.pt` (the checkpoint you'll use) and `weights/last.pt` (to resume with `resume=True`).
- `results.png` and `results.csv`: loss and metric curves for each epoch.
- Confusion matrix, precision-recall curves, and sample batch images with labels drawn on. Check the batch images first; they catch broken labels immediately.

Reading the curves

- Train loss falling while val loss rises means overfitting. Add data or augmentation, or stop earlier.
- Both losses flat and high means the model can't learn the task as set up. Check labels, `imgsz`, and class definitions.
- mAP still climbing at the final epoch means you should train longer.

10 Evaluating and improving

Goal: know what the metrics mean and how to fix what they reveal.

Metric	Meaning	If it's low
Precision	Of what the model predicted, how much was correct	Too many false alarms: add background images, raise <code>conf</code>
Recall	Of what really exists, how much the model found	Missing objects: more varied examples, larger <code>imgsz</code> , lower <code>conf</code>
mAP50	Accuracy when a box only needs 50% overlap	The model doesn't find the object at all
mAP50-95	Averaged over stricter overlap thresholds	Objects are found but boxes are loose: check label tightness

```
metrics = YOLO("runs/exp1/weights/best.pt").val(data="data.yaml")
print(metrics.box.map50, metrics.box.map)    # mAP50, mAP50-95
```

Copy

What to try, in order of payoff

1. Look at failure images and fix labels. This is almost always the biggest win.
2. Add data covering the cases the model gets wrong.
3. Increase `imgsz` for small objects.
4. Try a larger model size.
5. Adjust augmentation. For example, turn off `flip1r` when left/right orientation matters, as with text.

6. Tune hyperparameters with `model.tune(...)`. This is expensive, so do it last.

11 Export and deploy

Goal: run your model fast on the hardware you actually have.

Copy

```
model = YOLO("best.pt")
model.export(format="onnx") # most portable
model.export(format="engine", half=True) # NVIDIA TensorRT, FP16
model.export(format="openvino", int8=True, data="data.yaml") # Intel, quantized

# exported models load with the same API
YOLO("best.onnx")("page.png")
```

Target	Format
General CPU server, cross-platform	<code>onnx</code>
NVIDIA GPU or Jetson	<code>engine</code> (TensorRT)
Intel CPU, GPU, or NPU	<code>openvino</code>
iPhone, iPad, Mac	<code>coreml</code>
Android and microcontrollers	<code>tflite</code> (LiteRT)
Mobile and ARM CPUs	<code>ncnn</code>
Browser	<code>tfjs</code>

Key export arguments are `half` (FP16), `int8` (quantization, which needs `data` for calibration), `dynamic` (variable input sizes), `imgsz`, and for YOLO26, `nms`. After exporting, always re-run `val` on the exported file, because quantization can reduce accuracy.

For serving, the Guides section covers Docker, Triton, and specific devices such as Raspberry Pi and Jetson. The Rust inference library runs exported models without Python at all.

12 Tracking and Solutions

Goal: handle video, and know when not to build something yourself.

```
model = YOLO("yolo26n.pt")
for r in model.track(source="gate.mp4", persist=True, stream=True):
    if r.bboxes.id is not None:
        ids = r.bboxes.id.int().tolist()    # same object keeps the same ID across
```

Copy

Available trackers are TrackTrack (the default), BoT-SORT, ByteTrack, OC-SORT, Deep OC-SORT, and FastTracker. You select one with `tracker="bytetrack.yaml"`, for example.

Solutions are finished applications built on top of this: object counting across a line or region, heatmaps, queue management, speed estimation, security alarms, workout monitoring, and more. Check the Solutions page before building a video-analytics feature; one may already cover most of it.

13 Platform, integrations, and the LLM interface

Goal: know what exists outside the core package.

- **Ultralytics Platform:** annotation, dataset management, cloud training, and deployment in the browser. Useful if you'd rather not manage GPUs, or if non-developers need to label data.
- **Integrations:** experiment tracking (such as Weights & Biases, MLflow, Comet, TensorBoard), dataset tools, and deployment targets. Most integrations switch on automatically once the relevant package is installed.
- **LLM interface:** an OpenAI-compatible wrapper for language and vision models that pairs with YOLO pipelines. For example, YOLO crops a region and an LLM interprets it.

- **Reference:** API documentation generated from the source code. Use it when you need exact method signatures.

14 Licensing, before you ship anything

Goal: avoid building a commercial product on terms you can't meet.

license risk 0.99

AGPL-3.0 is the default license. It's a strong copyleft license: if you distribute software that uses Ultralytics, or let users interact with it over a network (a SaaS product or client-facing web service), you generally have to release that software's source code under AGPL too. That covers your application, not just the model.

Enterprise License: Ultralytics sells this for business use, including internal tools, automated workflows, and production deployments, and it removes the AGPL obligations.

This applies to models trained with Ultralytics code as well. For a paid product or client work, get a license or legal advice before building on it. This guide isn't legal advice.

Learning, personal projects, and open-source work are fine under AGPL.

15 Capstone: title-block detection on drawing sheets

Goal: tie steps 3 to 11 together on a real problem, finding sheet number and title regions on architectural drawing pages.

Framing it correctly

YOLO returns the *region*. Reading the text inside it is a separate step:

`page.get_text(clip=...)` in PyMuPDF for vector PDFs, or OCR or a vision LLM for

scanned pages. In a pipeline like that, YOLO would take over from a Roboflow-hosted detector, not from the text extraction.

Plan

1. **Task:** detection, with classes `title_block`, `sheet_number`, `sheet_title`. If some sheets are rotated, try OBB.
2. **Data:** export existing annotations from Roboflow in YOLO format. Split by PDF, not by page, so pages from one drawing set never appear in both train and val. Add sets with different layouts, since layout variety was the known weak spot.
3. **Resolution:** sheet numbers are tiny on a full sheet, so start with `imgsz=1280`. If recall on `sheet_number` is still low, try the `yolo26s-p2.yaml` small-object config, or detect the title block first and then run a second pass on the crop.
4. **Augmentation:** set `flipplr=0`, because mirrored text isn't realistic.
5. **Train** `yolo26s.pt`, check the batch images, and read the confusion matrix. Is `sheet_title` being confused with neighboring fields?
6. **Map back to PDF space:** render each page at a known DPI, and convert the normalized box to PDF points using the page width and height.
7. **Export** to ONNX for a CPU server, and re-run `val` on the exported file.

```
import cv2, numpy as np
import fitz                                     # PyMuPDF
from ultralytics import YOLO

model = YOLO("best.onnx")
doc = fitz.open("set.pdf")
for page in doc:
    pix = page.get_pixmap(dpi=100)
    img = pix.tobytes("png")
    arr = cv2.imdecode(np.frombuffer(img, np.uint8), cv2.IMREAD_COLOR)
    r = model(arr, imgsz=1280, conf=0.4)[0]
    W, H = page.rect.width, page.rect.height
    for b in r.bboxes:
        x1, y1, x2, y2 = b.xyxy[0].tolist()
        clip = fitz.Rect(x1*W, y1*H, x2*W, y2*H)
        label = r.names[int(b.cls[0])]
        print(page.number, label, page.get_text(clip=clip).strip())
```

Copy

Before committing, compare this against what you already have. If coordinate reuse plus a vision LLM on the first pages already works for most sets, a YOLO model mainly helps on unusual layouts and large volumes. Measure where the current pipeline fails before investing in training. Also note step 14: a client-facing extraction service built on this falls under AGPL unless licensed.

16 Roadmap and checklist

Goal: a realistic pace, and a way to confirm you actually learned it.

Week	Focus	Docs sections
1	Steps 1–6: install, predict, all seven tasks on sample images, parsing results	Quickstart, Tasks, Modes → Predict
2	Steps 7–9: pick a small dataset, label about 200 images, first training run	Models, Datasets, Modes → Train
3	Step 10: error analysis, two or three improvement iterations	Modes → Val, Guides (model evaluation, hyperparameter tuning)
4	Steps 11–12: export, benchmark, tracking or Solutions	Modes → Export / Benchmark / Track, Solutions
5+	Step 15 capstone, and read about licensing before any production use	Guides for your deployment target, Integrations

You've learned it when you can

- ☐ Name the right task and model family for a new problem without looking anything up.
- ☐ Build a correct YOLO-format dataset with a leak-free split.
- ☐ Train, then explain from the curves and confusion matrix why the model fails where it does.
- ☐ Make a change that measurably improves mAP50-95, and explain why it worked.

- ☐ Export to the right format, verify accuracy after export, and measure latency on the target machine.
- ☐ Explain the AGPL obligations to a client in two sentences.

Based on docs.ultralytics.com (Home, Models, and YOLO26 pages) as of September 16, 2026. Benchmark figures are Ultralytics' own. Check the live docs for anything version-specific, since the library updates frequently.